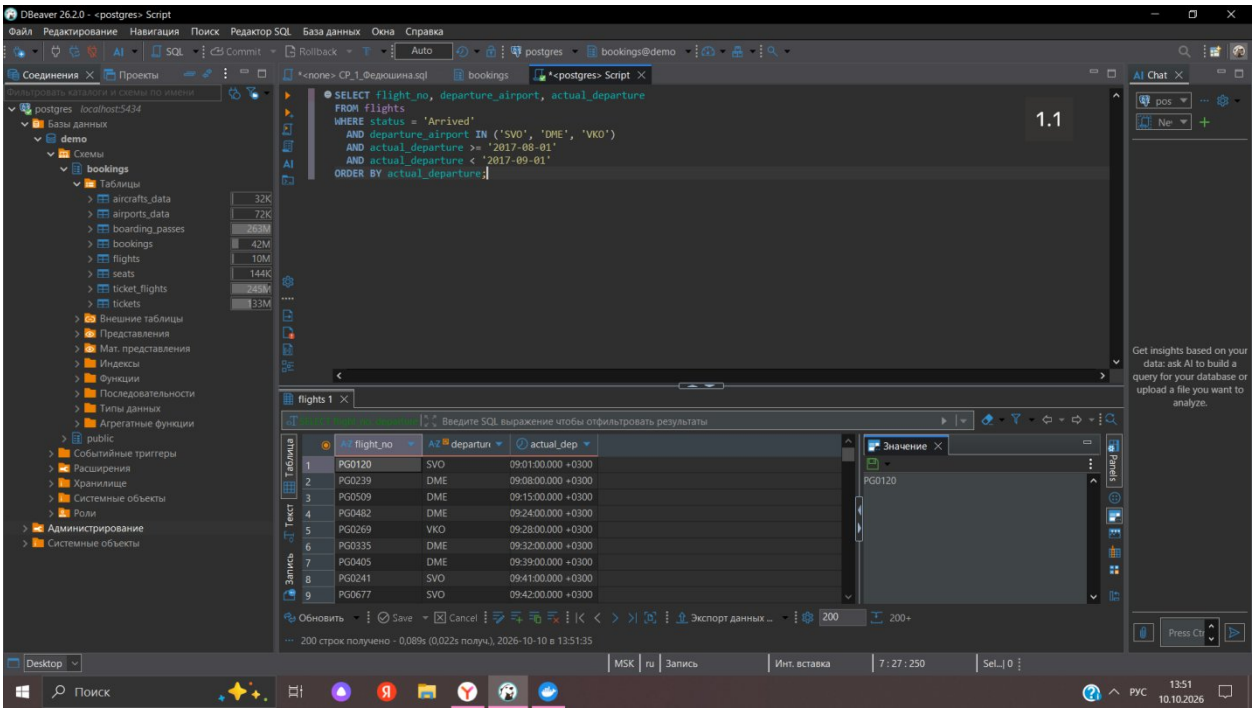
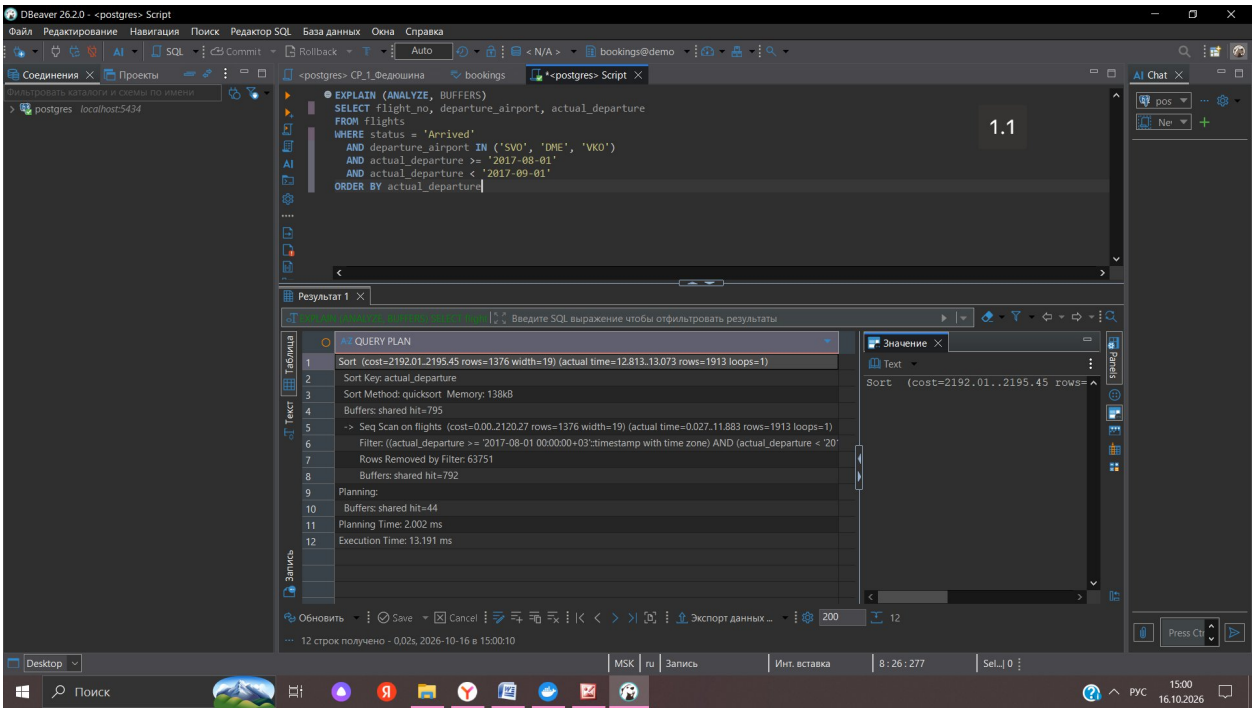
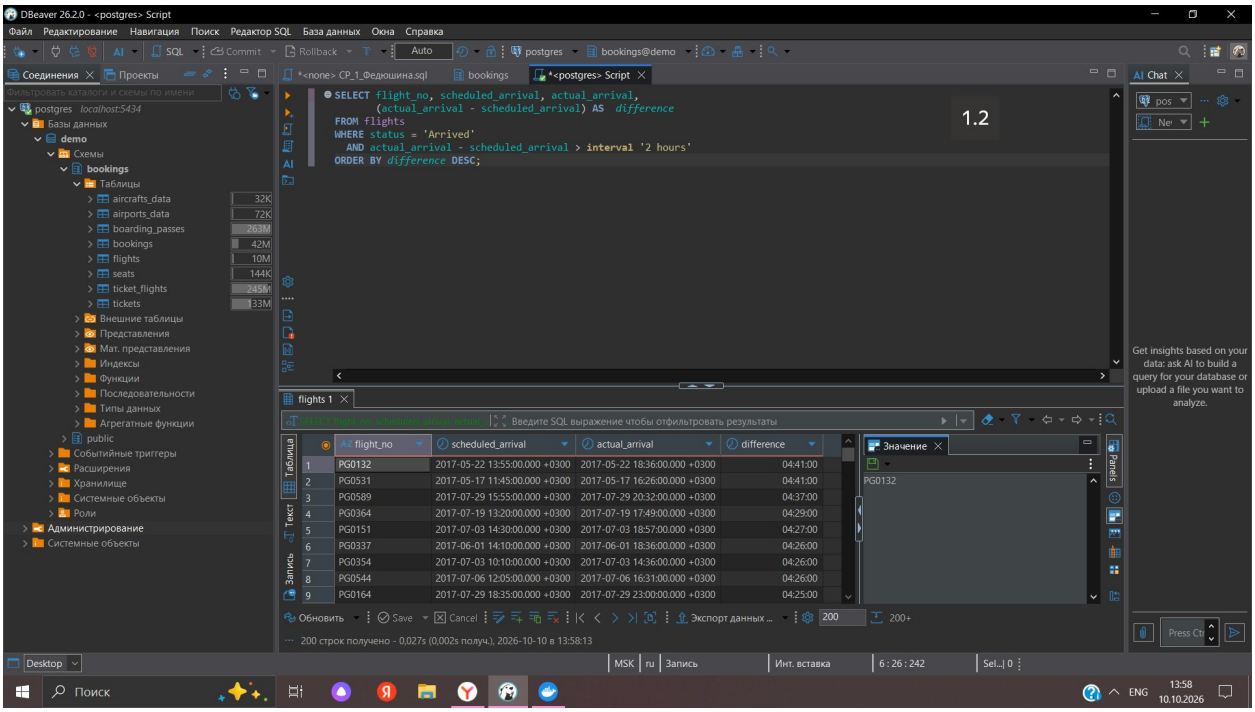
Задание 1.1

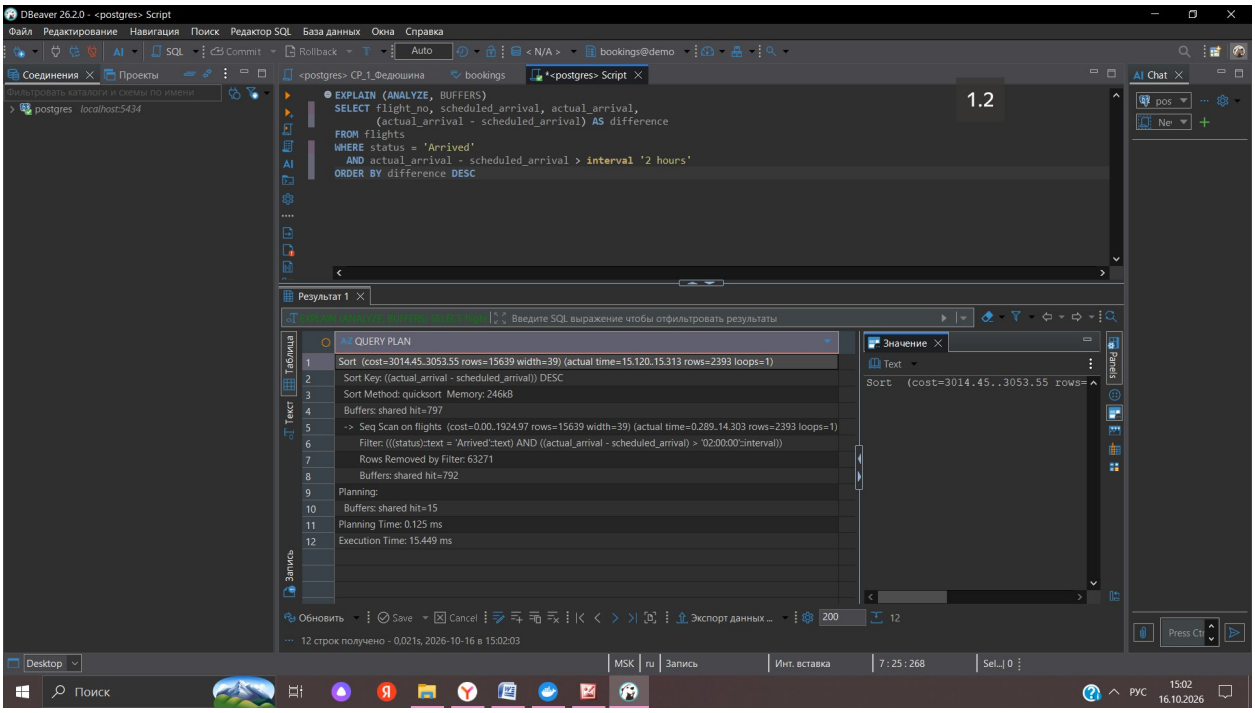




Запрос выполнен через Seq Scan on flights + Sort, индекс не использовался, все неподходящие строки отфильтрованы после полного сканирования таблицы. Работа шла полностью с кешем (shared hit=795, shared read=0), сортировка поместилась в память (quicksort, 138 kB), время выполнения — 13.191 мс.

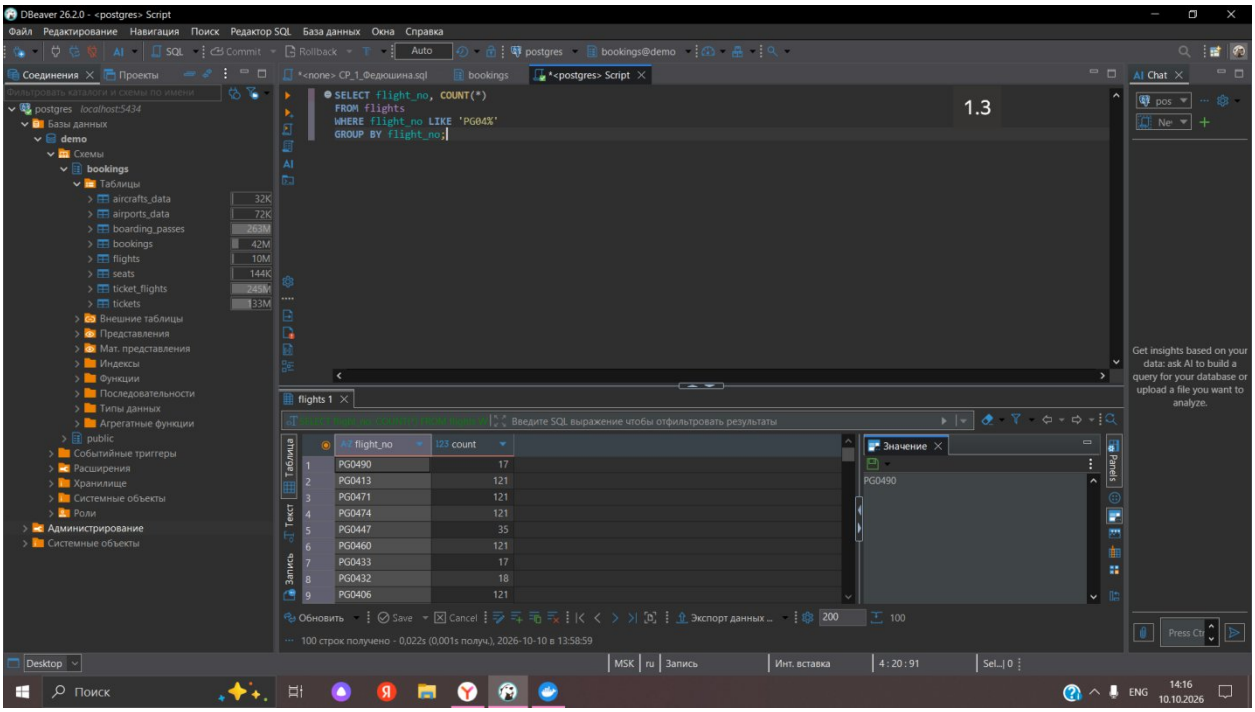
Задание 1.2

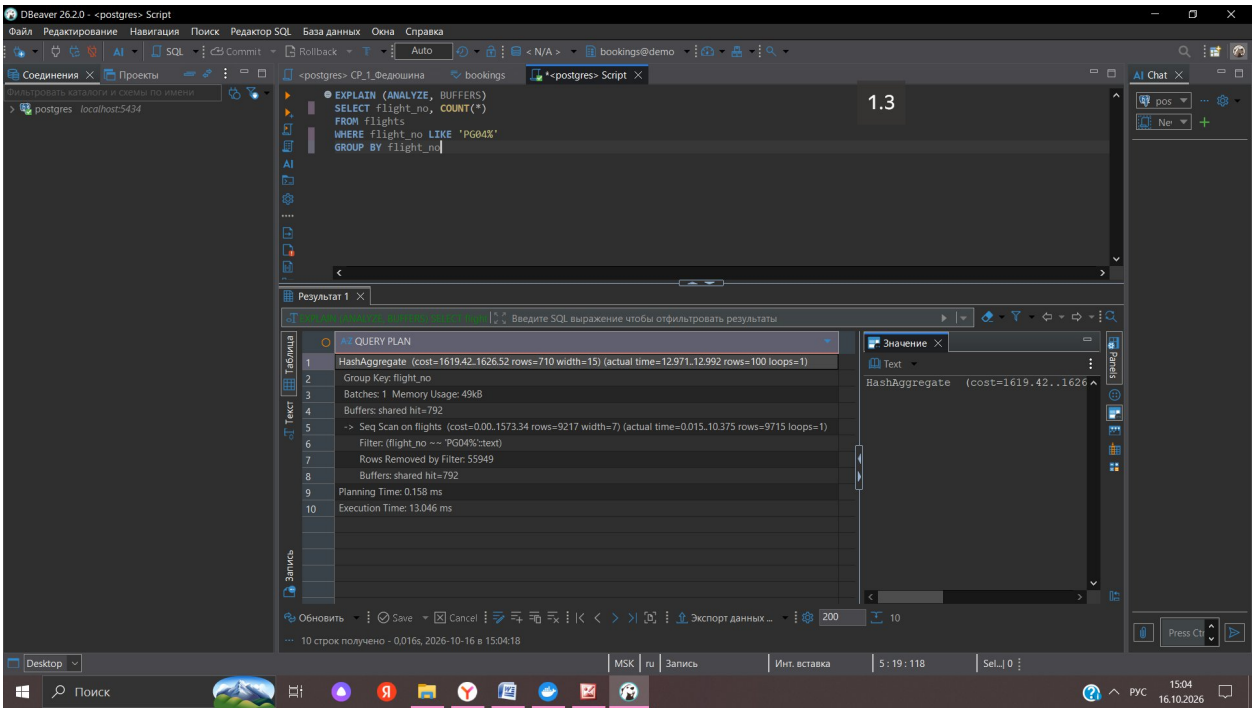




Запрос выполнялся через Sort + Seq Scan on flights. Индекс не использовался, в плане только Seq Scan. Работа шла полностью с кешем: shared hit=797, shared read=0, обращений к диску не было. Сортировка поместилась в память (Sort Method: quicksort, Memory: 246kB), внешние временные файлы не использовались; время выполнения — 15.449 мс, результат — 2393 строки.

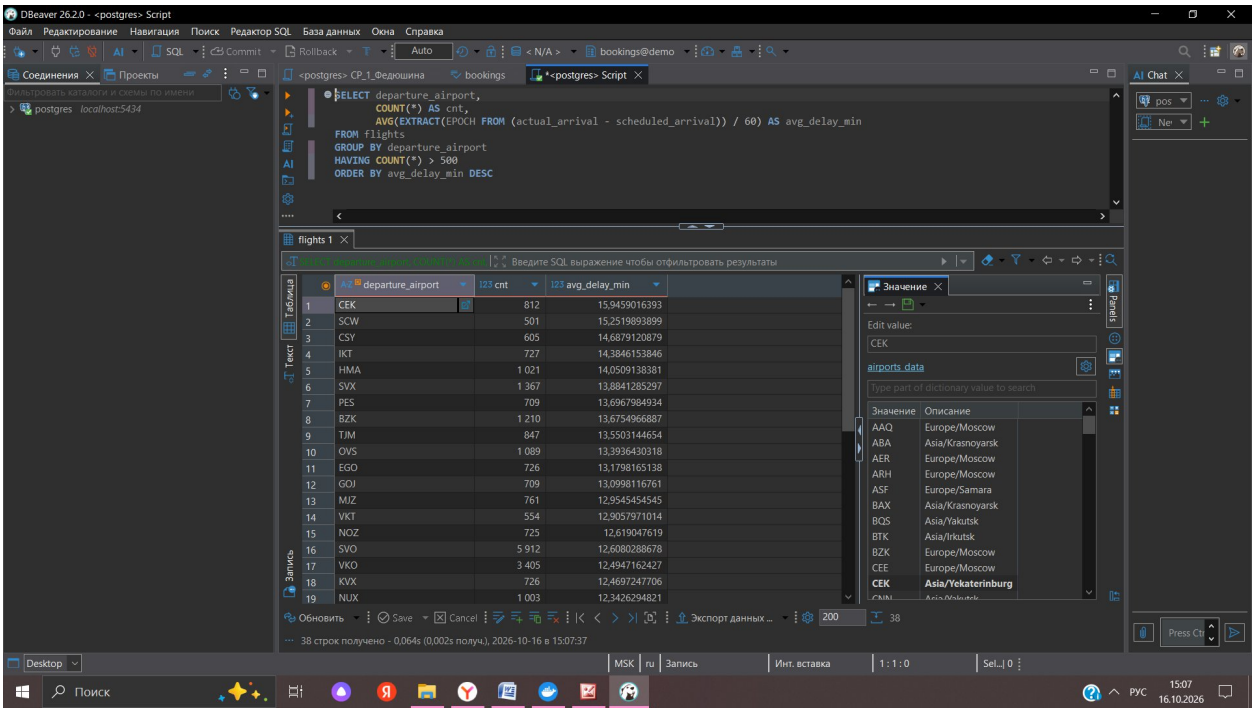
Задание 1.3

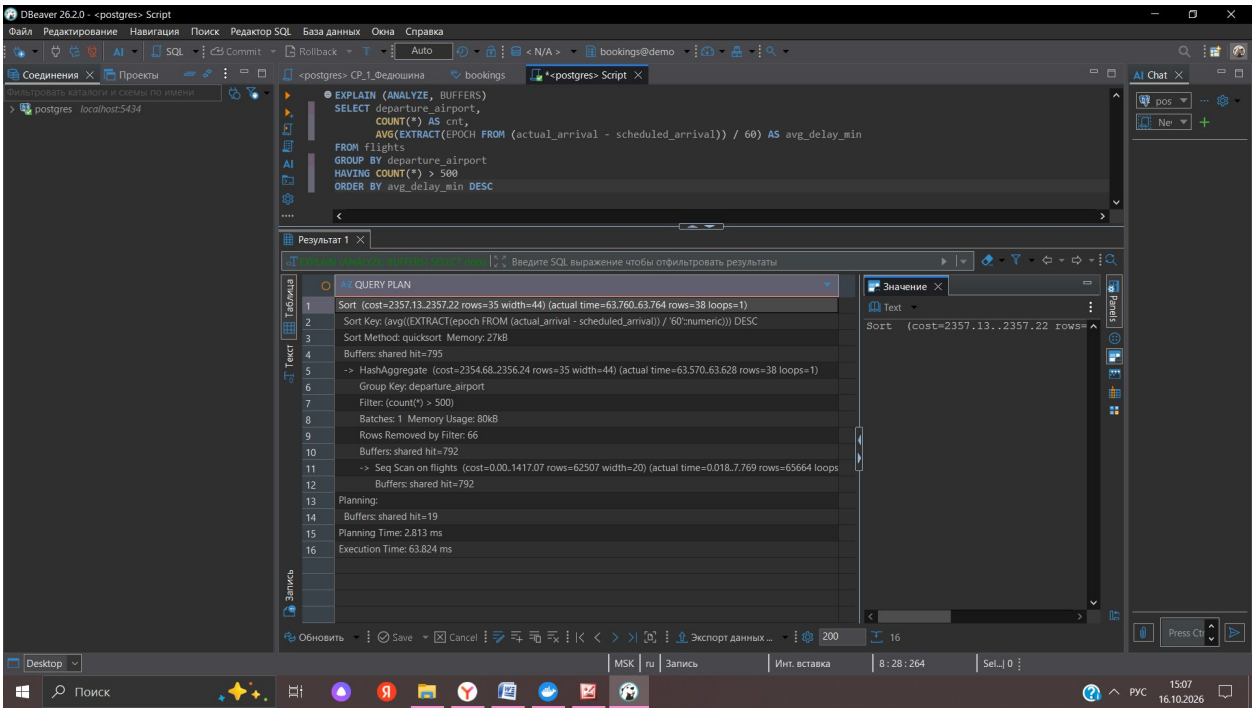




Запрос выполнялся через HashAggregate + Seq Scan on flights. Индекс не использовался. Работа шла полностью с кешем: shared hit=792, shared read=0, обращений к диску не было. Время выполнения — 13.046 мс.

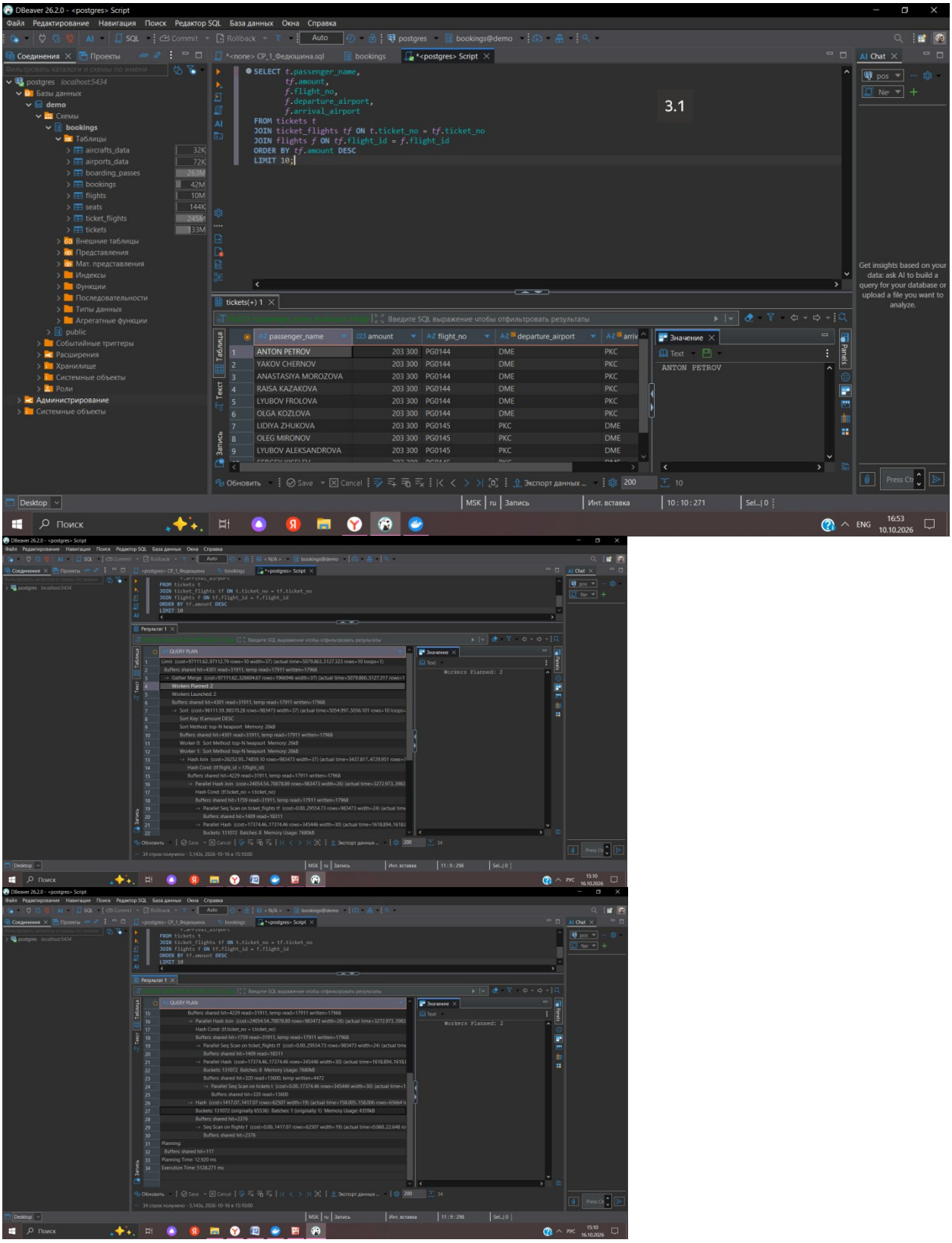
Задание 2.1





Запрос выполнялся через Sort + HashAggregate + Seq Scan on flights. Индекс не использовался**.** HAVING отражён как Filter: (count(*) > 500) в узле HashAggregate и применяется **после** агрегации: из 104 аэропортов отброшено 66 с малым числом рейсов, осталось 38. Работа шла полностью с кешем: shared hit=795, shared read=0 — обращений к диску не было, сортировка поместилась в память (quicksort, 27kB), время выполнения — 63.824 мс

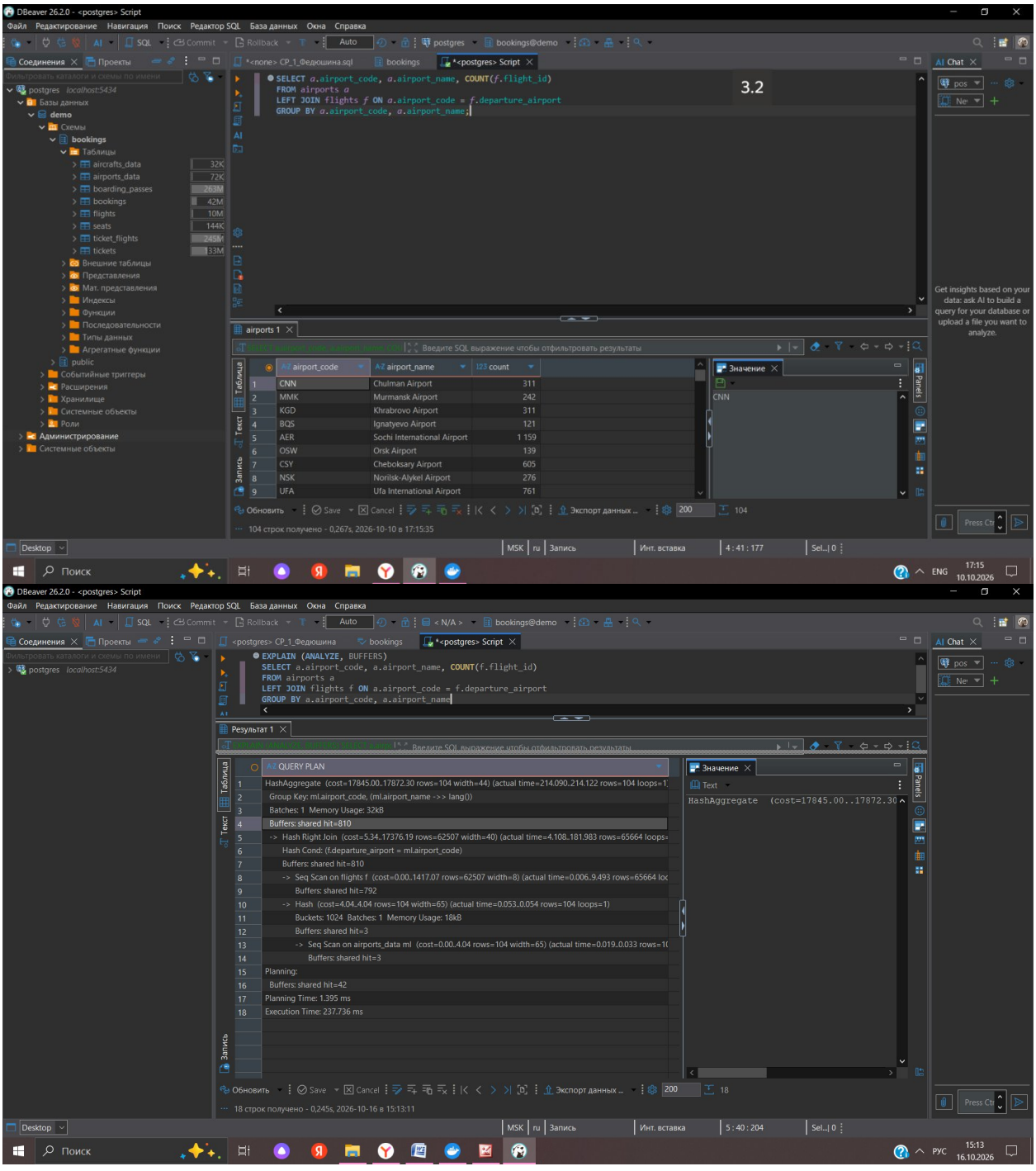
Задание 3.1



Запрос выполнялся через Limit + Gather Merge + Sort (top-N heapsort) + два Hash Join + Parallel Seq Scan по tickets, ticket_flights и flights. Индексы не использовались, все три таблицы читались целиком. Запрос работал с диском: shared read=31911 значительно превышает shared hit=4301, плюс зафиксированы temp read/written сортировка и

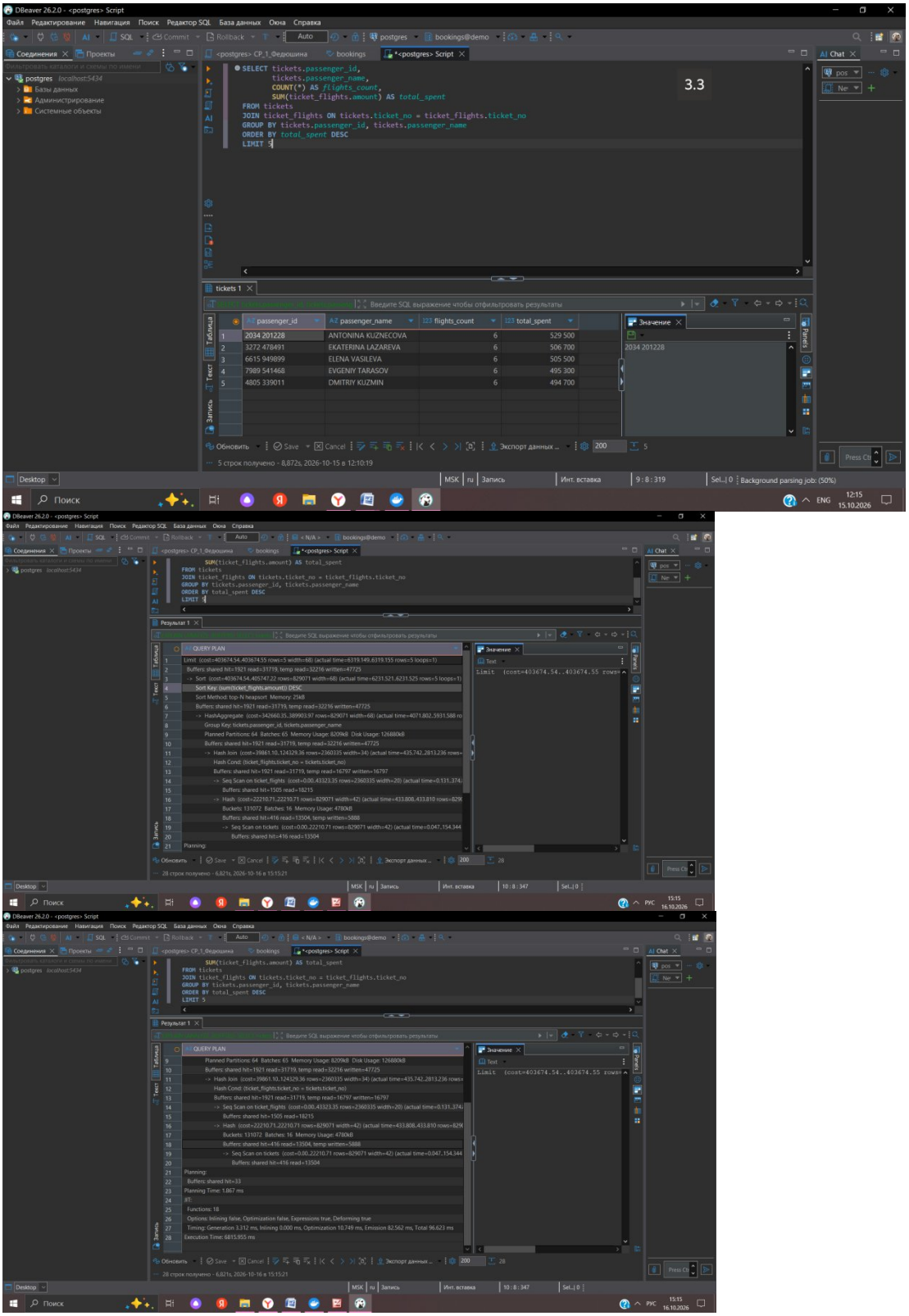
хеширование сбрасывались во временные файлы. Sort Method: top-N heapsort (26 kB) означает, что сортировка держала в памяти только 10 наибольших значений, а не всю таблицу

Задание 3.2



Запрос выполнялся через HashAggregate + Hash Right Join + два Seq Scan по airports_data и flights. Индекс не использовался. Работа шла полностью с кешем: shared hit=810 во всех узлах, shared read=0 — обращений к диску не было. Hash Right Join строит хеш по маленькой airports_data (shared hit=3, 104 строки, 18 kB), затем однократно сканирует большую flights (shared hit=792, 65 664 строки). Время выполнения — 237.736 мс, результат 104 строки.

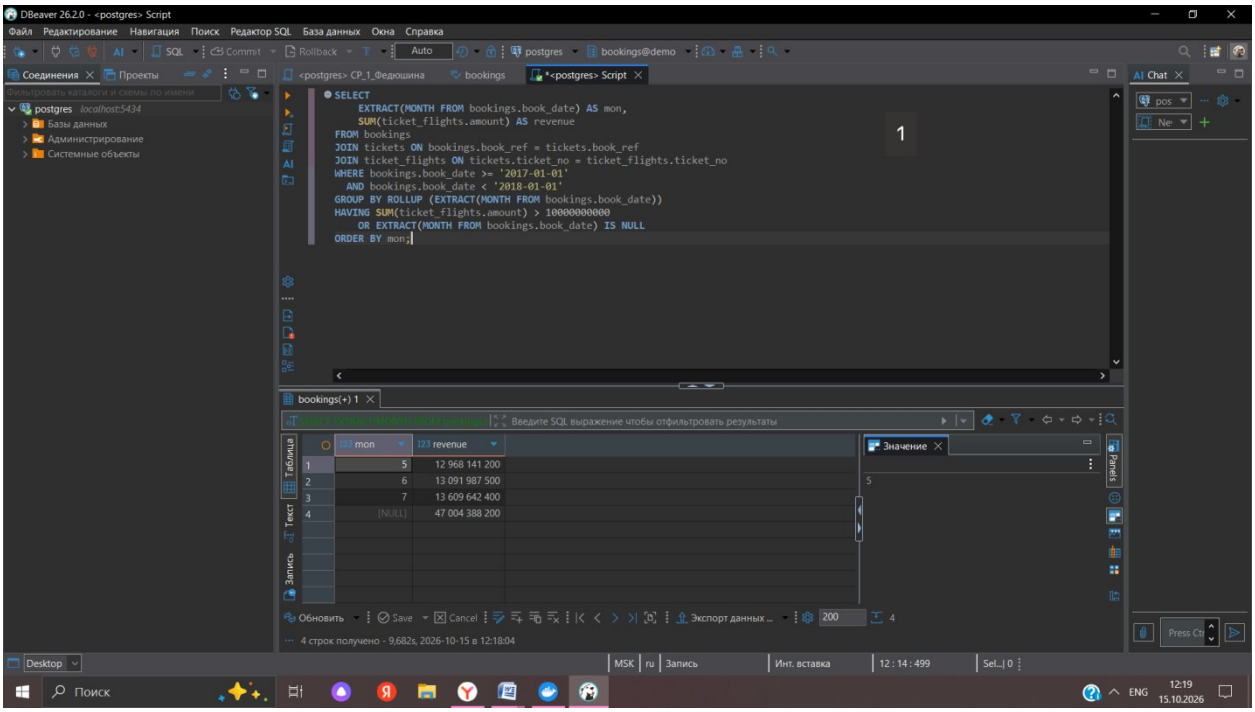
Задание 3.3

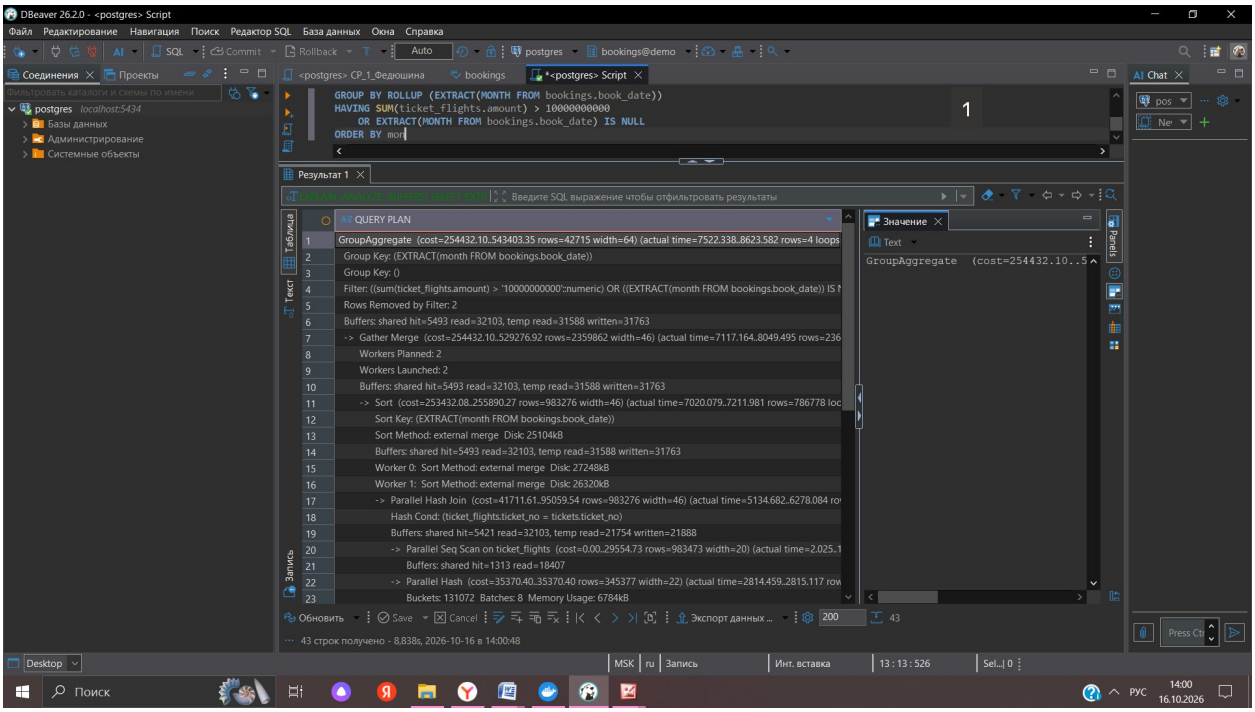


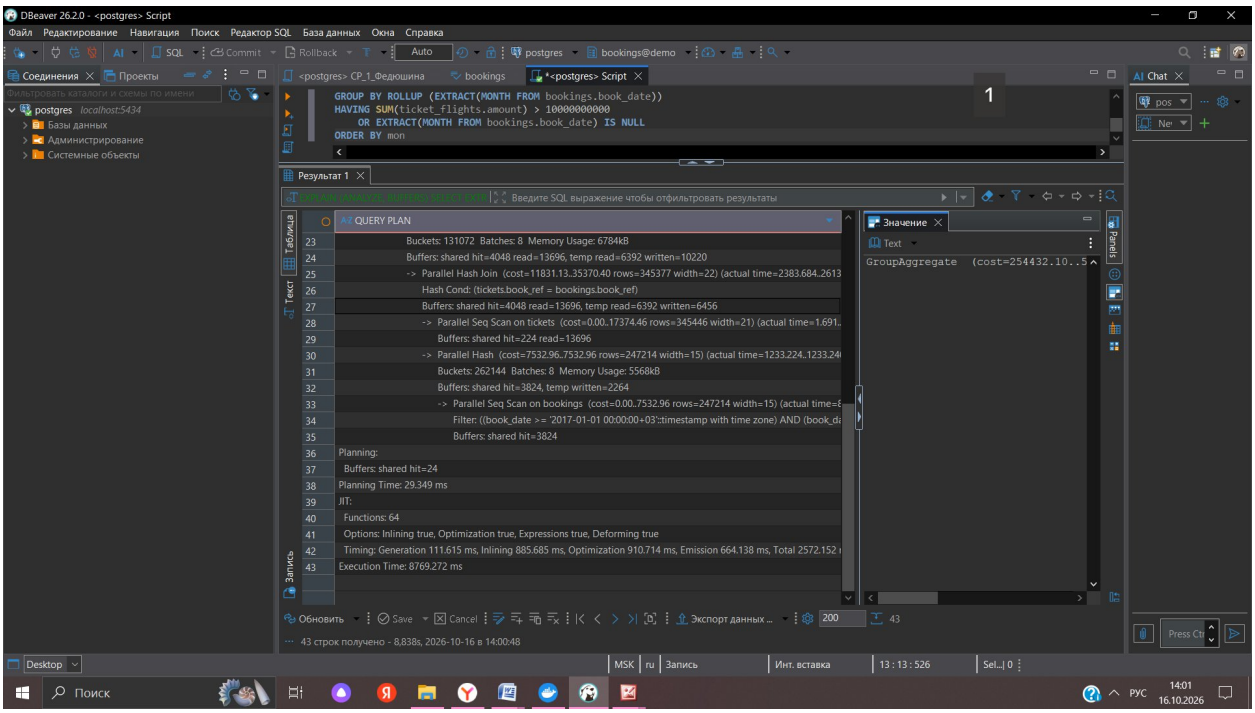
Запрос выполнялся через Limit + Sort (top-N heapsort) + HashAggregate + Hash Join + два Seq Scan по tickets и ticket_flights. Индекс не использовался. Запрос работал с диском: shared read=31719 , а HashAggregate не поместился в память (Planned Partitions:

64, Batches: 65, Disk Usage: 126880kB) и сбрасывался во временные файлы (temp read=32216 written=47725). Из-за большого объёма данных и работы с диском время выполнения составило 6 816 мс; top-N heapsort (25kB) позволил держать в памяти только 5 наибольших значений.

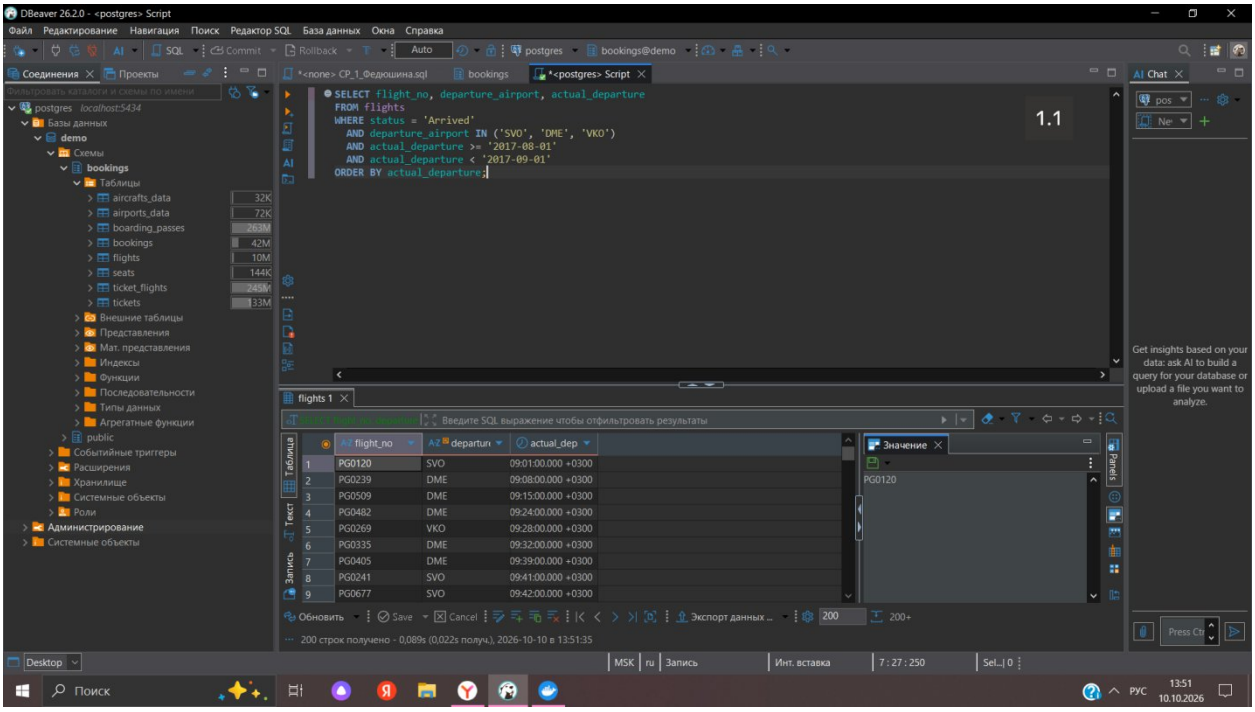
Задание 1





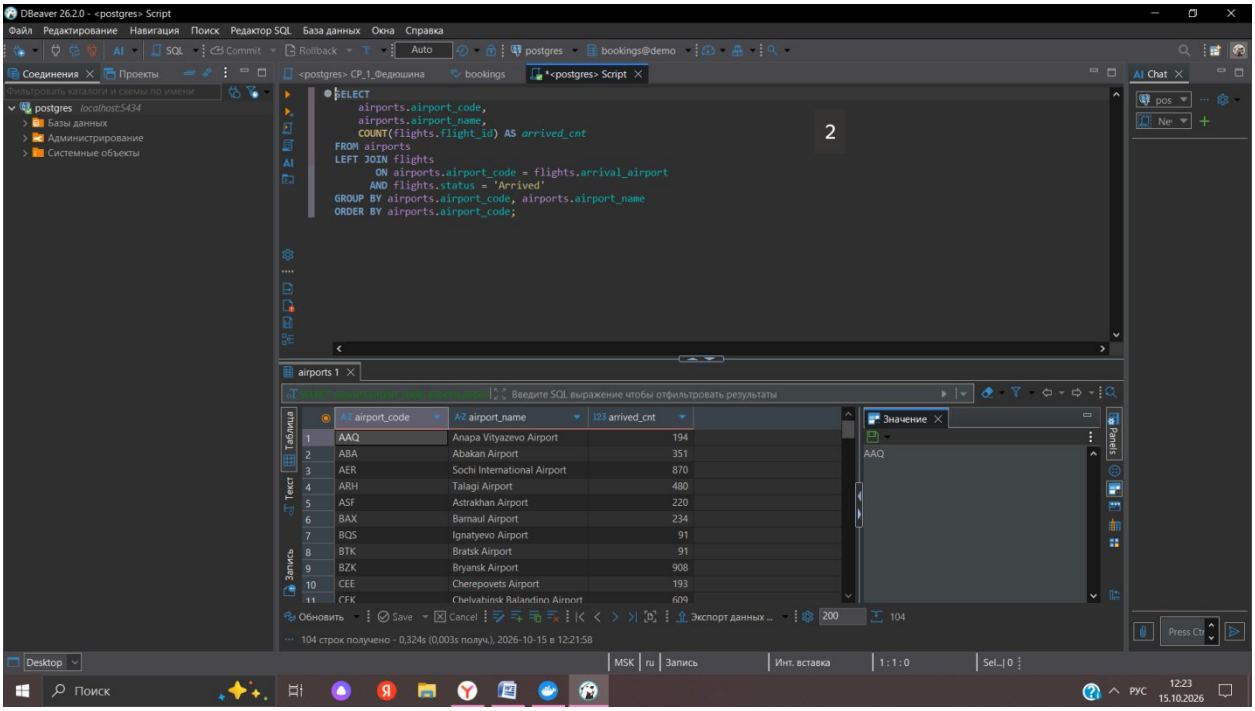


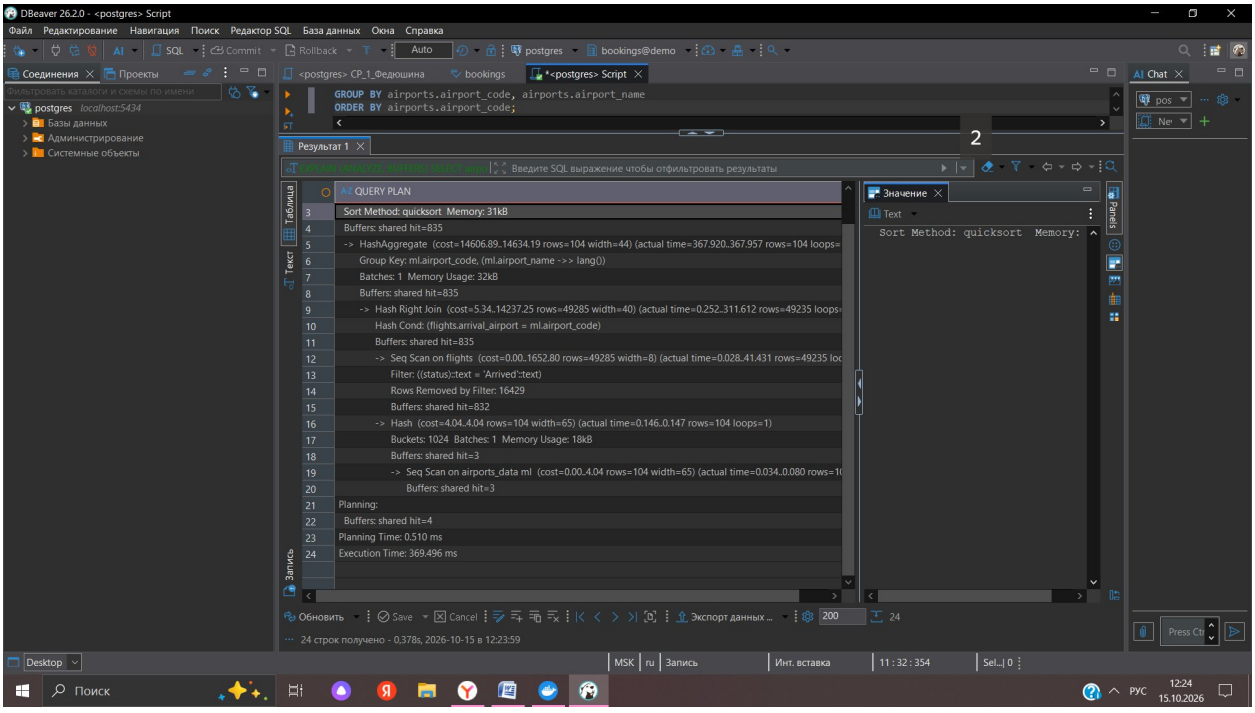
Запрос выполнялся через Parallel Hash Join и GroupAggregate с ROLLUP, без использования индексов — все три таблицы читались через Seq Scan. Основная работа шла с диском: shared read и temp read/written значительно превышают shared hit, то есть данные не умещались в кеш и частично сбрасывались во временные файлы.



Задание 2

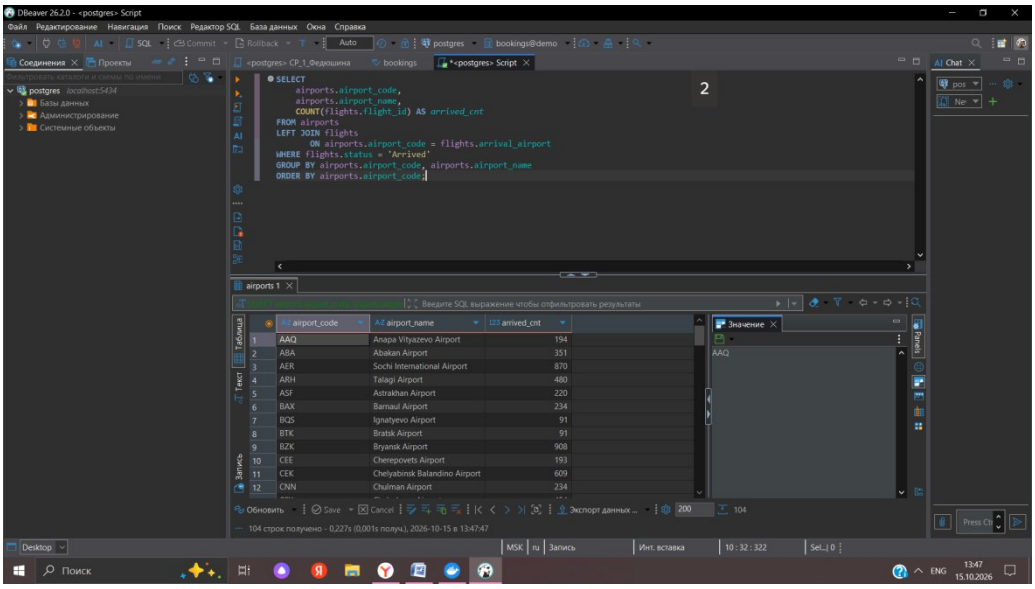
Условие по статусу перенесите в ON соединения, а не в WHERE

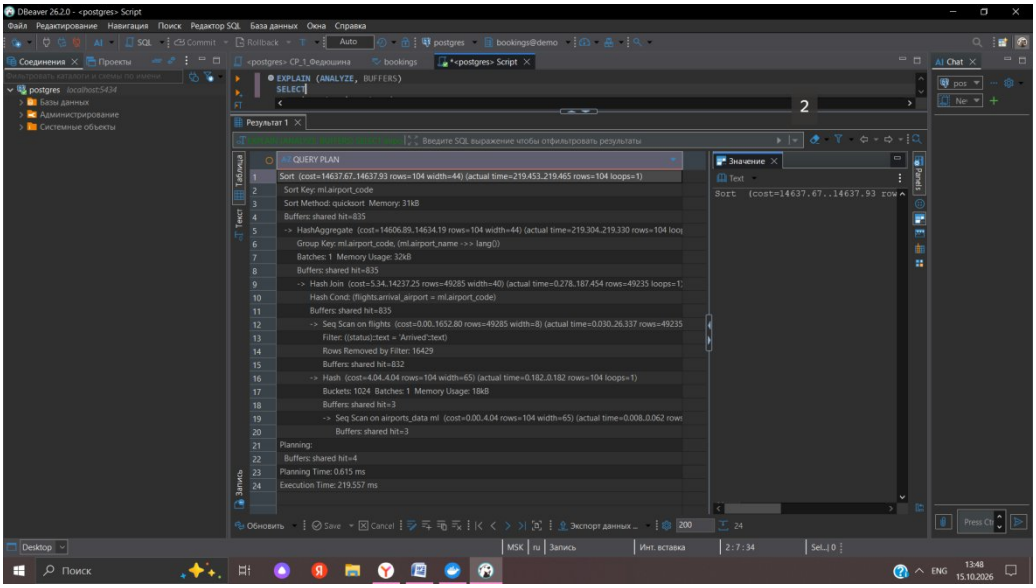




Запрос выполняется через Hash Right Join: хеш строится по маленькой таблице airports, затем однократно сканируется flights с фильтром status = 'Arrived'. Индекс не использовался, оба сканирования Seq Scan, что видно по узлам Seq Scan on flights и Seq Scan on airports m1. Работа шла полностью с кешем: shared hit=835, shared read=0 во всех узлах, обращений к диску не было. LEFT JOIN сохраняет все аэропорты, включая те, куда не прилетал ни один рейс, и COUNT(flights.flight_id) корректно даёт для них 0

Условие в Where



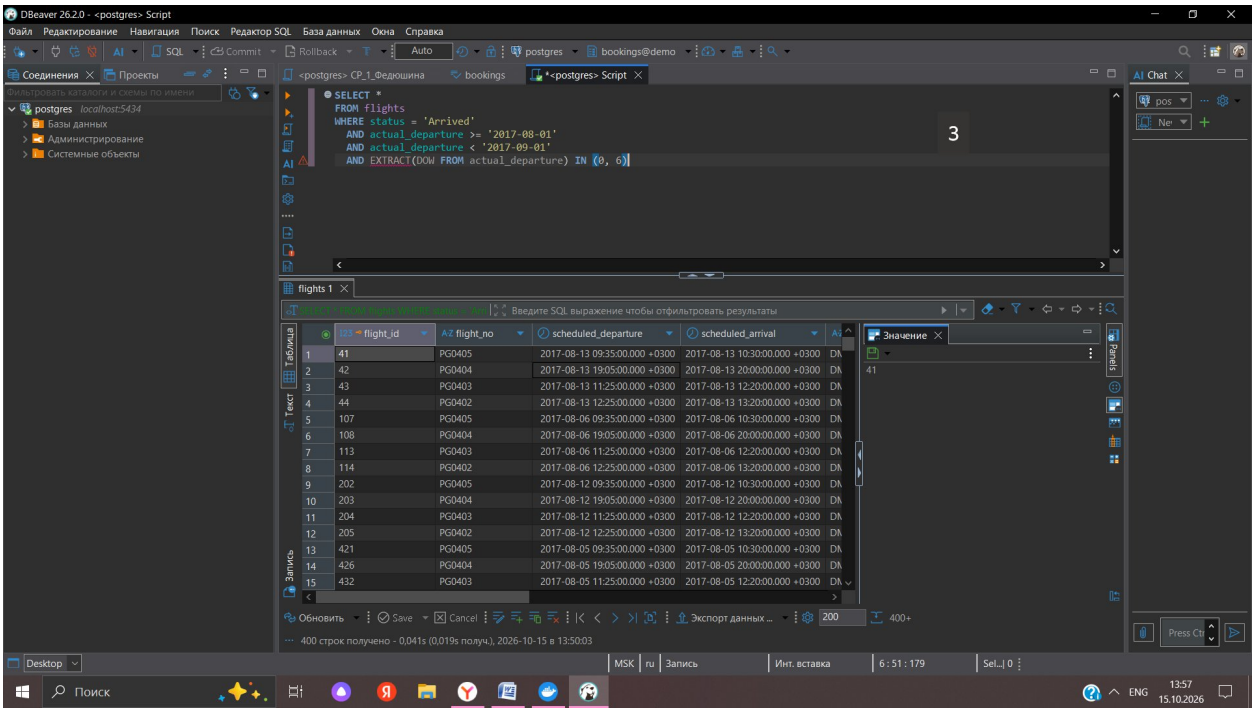


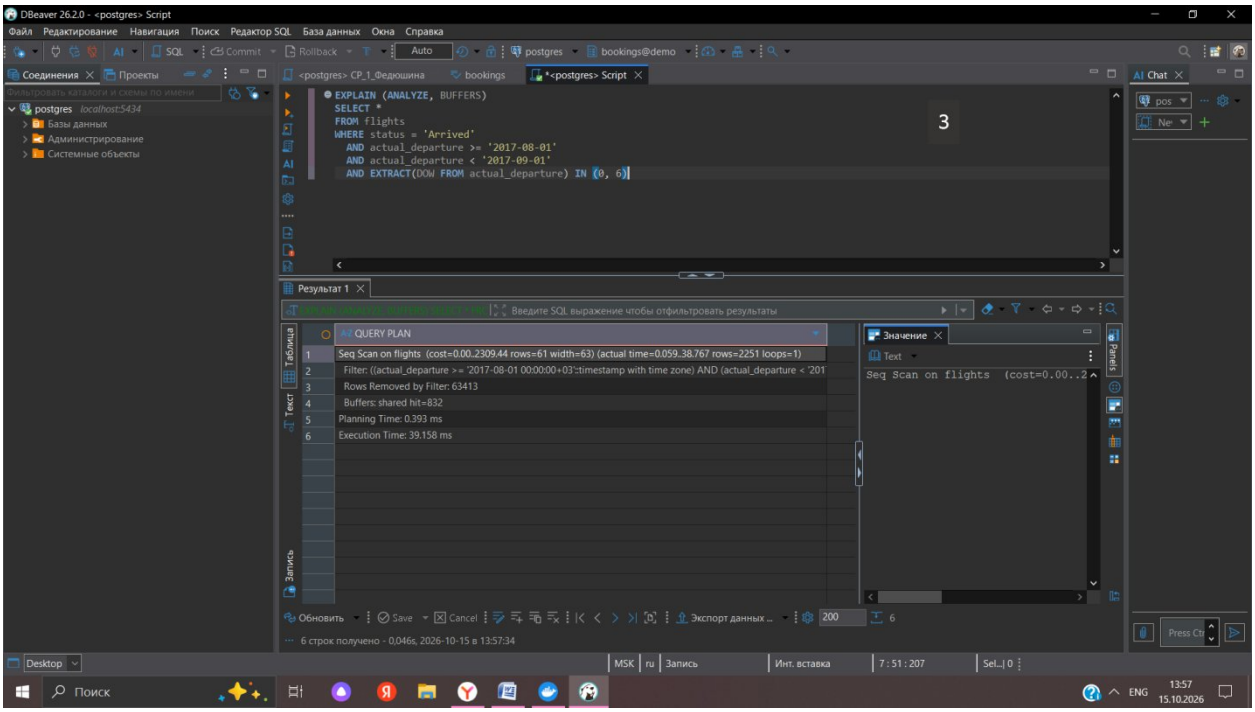
При переносе условия status = 'Arrived' в WHERE узел соединения сменился на Hash Join без Right, то есть LEFT JOIN фактически превратился во внутренний INNER JOIN. Аэропорты без прибытий исчезли из результата, поэтому время выполнения оказалось меньше (219 мс против 369 мс) — но это не оптимизация, а потеря строк. Индекс по-прежнему не использовался, работа также шла полностью с кешем (shared hit=835, shared read=0). Это доказывает, что условие по статусу обязано находиться в ON, а не в WHERE.

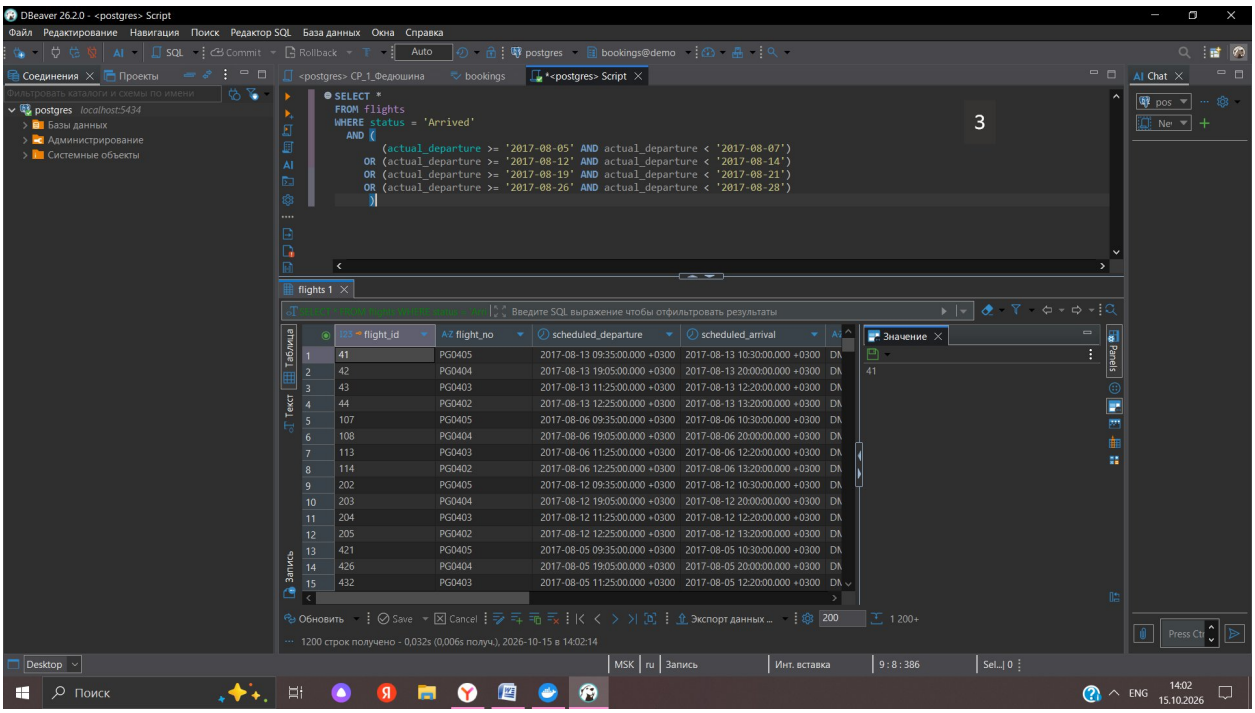
Почему COUNT(*) здесь недопустим. Если заменить на COUNT(*), то для аэропортов без прибытий LEFT JOIN даёт одну строку с NULL во всех столбцах flights. COUNT(*) посчитает эту NULL-строку как 1, и аэропорт без рейсов получит 1 вместо 0.

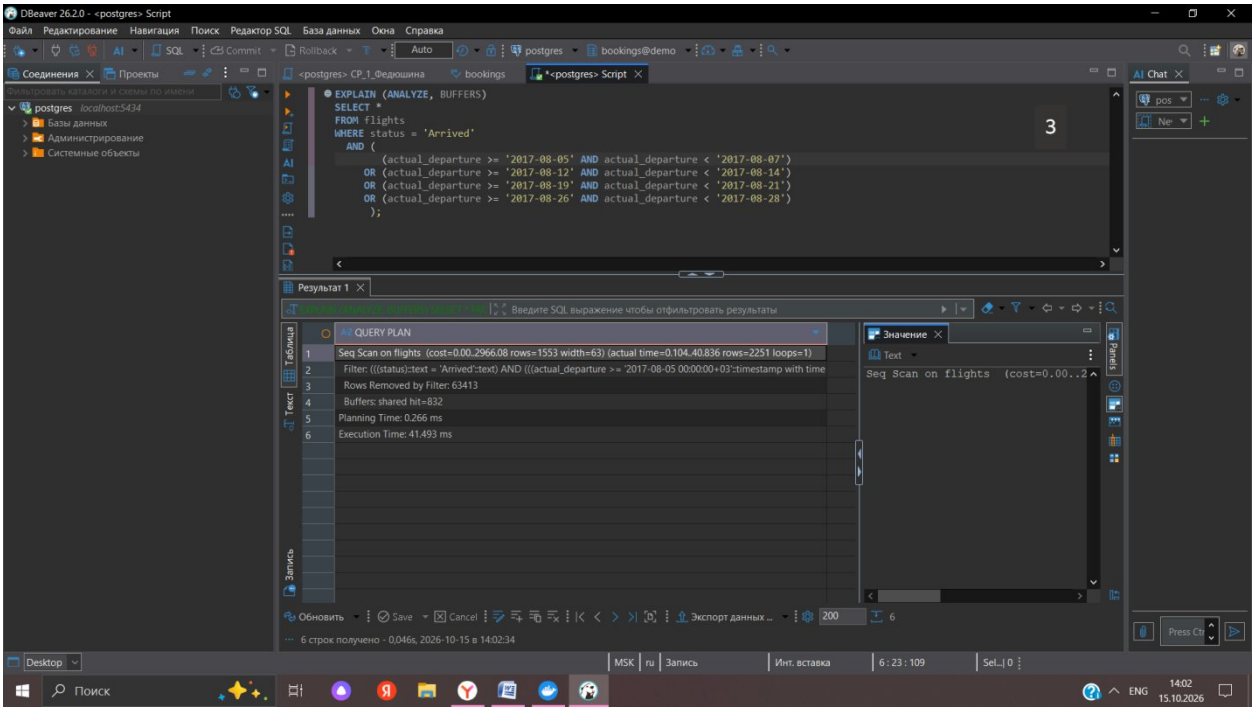
Hash Right Join — это соединение, при котором постгрис строит таблицу по правой таблице , а затем однократно сканирует левую и ищет совпадения в хеше.

Задание 3


DBeaver 26.2.0 - <postgres> Script
SELECT *
FROM flights
WHERE status = 'Arrived'
AND actual_departure >= '2017-08-01'
AND actual_departure < '2017-09-01'
AND EXTRACT(DOW FROM actual_departure) IN (0, 6)
3
flights 1
flight_id
flight_no
scheduled_departure
scheduled_arrival
400 строк получено - 0,041s (0,019s получ.), 2026-10-15 в 13:50:03


DBeaver 26.2.0 - <postgres> Script
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM flights
WHERE status = 'Arrived'
AND actual_departure >= '2017-08-01'
AND actual_departure < '2017-09-01'
AND EXTRACT(DOW FROM actual_departure) IN (0, 6)
3
Результат 1
QUERY PLAN
Seq Scan on flights (cost=0.00..2309.44 rows=61 width=63) (actual time=0.059..38.767 rows=2251 loops=1)
Filter: ((actual_departure >= '2017-08-01 00:00:00+03'::timestamp with time zone) AND (actual_departure < '201
Rows Removed by Filter: 63413
Buffers: shared hit=832
Planning Time: 0.393 ms
Execution Time: 39.158 ms
6 строк получено - 0,046s, 2026-10-15 в 13:57:34

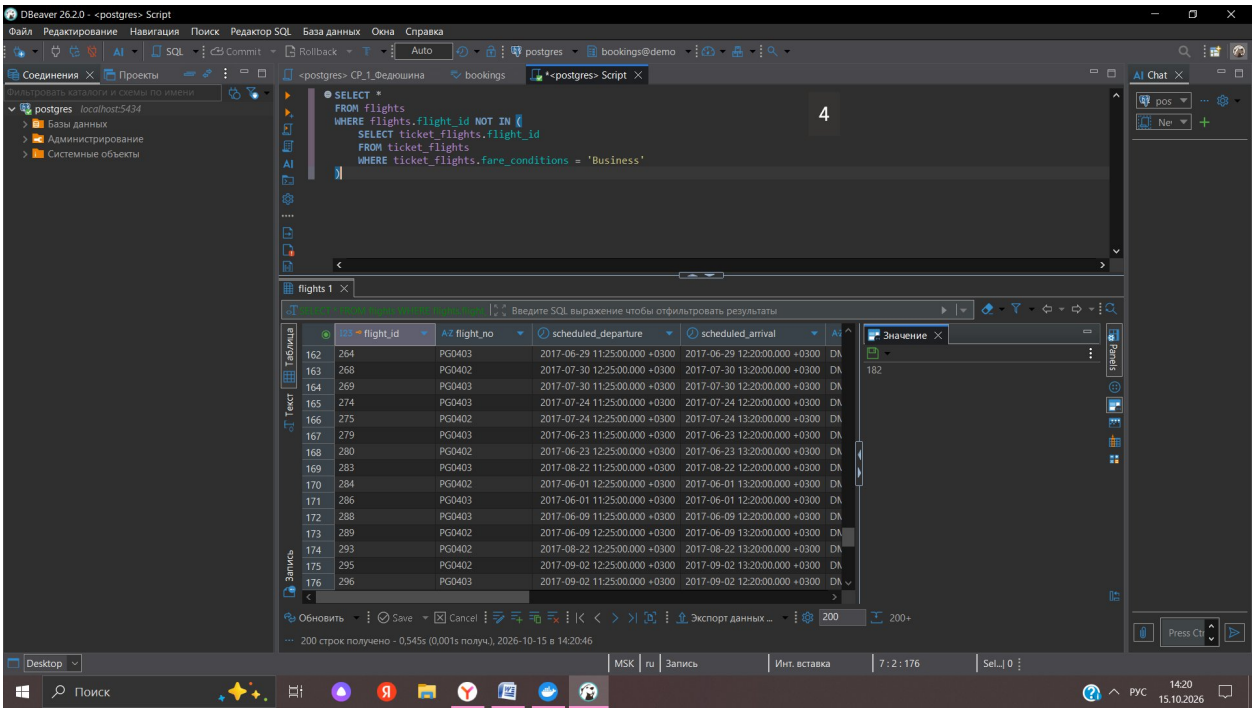


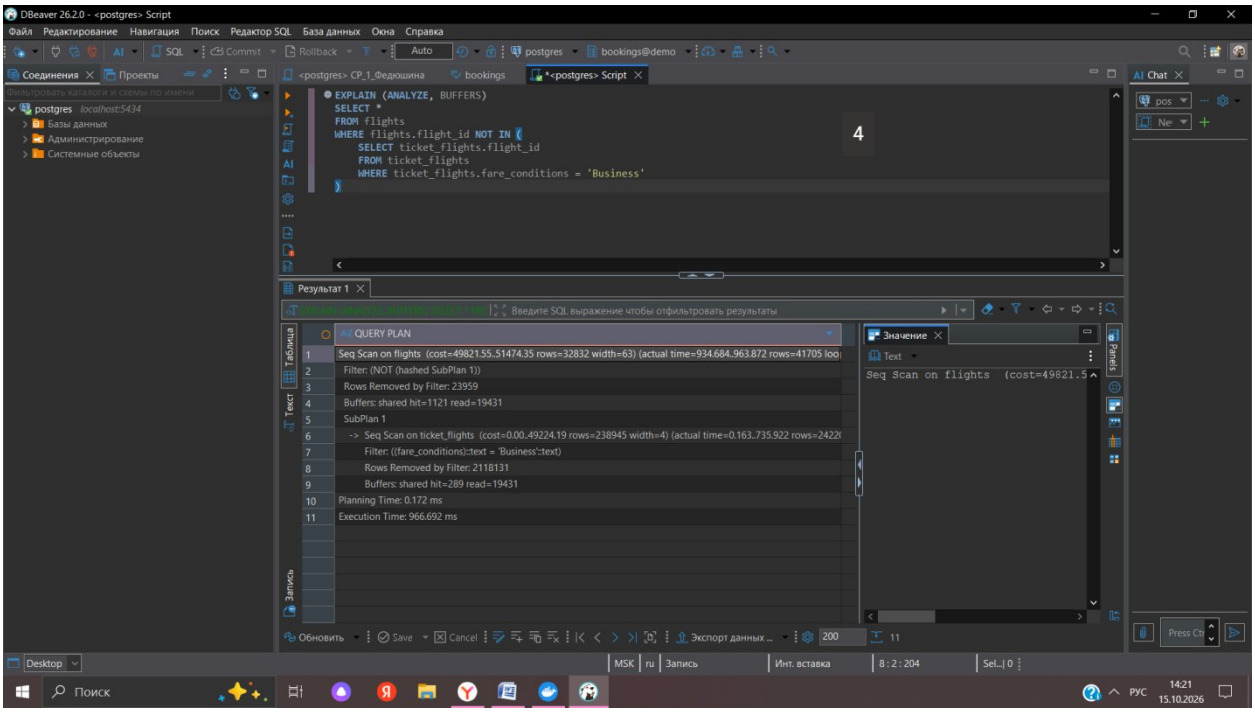


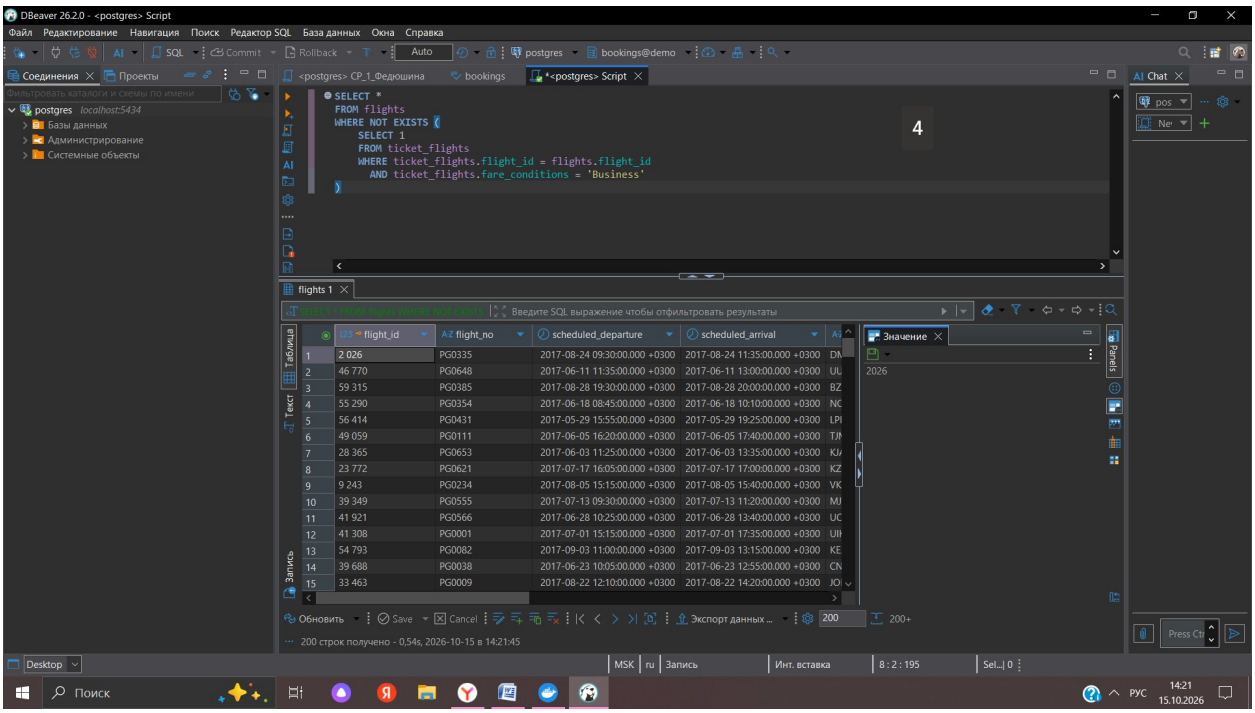
Оптимизированный оказался медленнее 41 мс и 39 мс. Потому что в оптимизированном варианте условие состоит из **четырёх веток.** Программа проверяет их по очереди для каждой строки и это чуть больше, чем одна проверка EXTRACT.

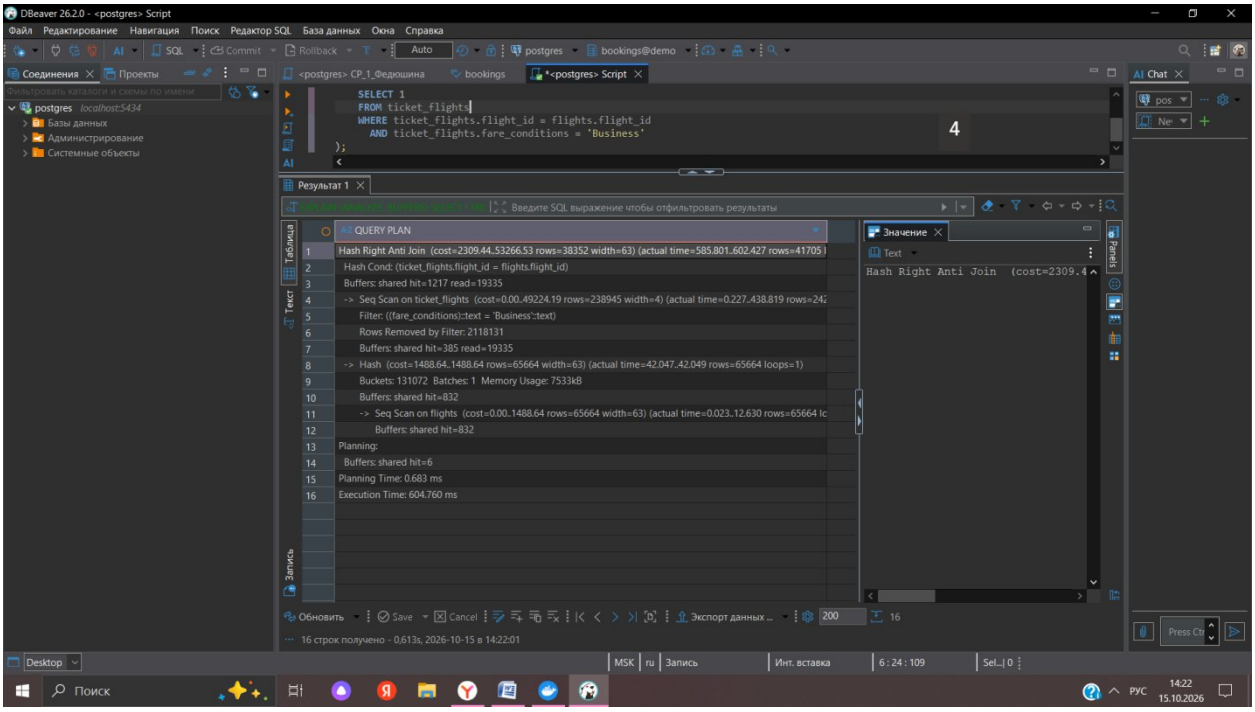
Оба запроса выполняются через Seq Scan on flights: в первом случае фильтр содержит EXTRACT(DOW ...), во втором — четыре диапазона дат. Индекс **не используется в обоих вариантах.** Работа идёт полностью **с кешем:** shared hit=832, shared read=0. Время почти одинаковое 39 мс и 41 мс

Задание 4


DBeaver 26.2.0 - <postgres> Script
SELECT *
FROM flights
WHERE flights.flight_id NOT IN (
SELECT ticket_flights.flight_id
FROM ticket_flights
WHERE ticket_flights.fare_conditions = 'Business'
)
4
flights 1
flight_id
flight_no
scheduled_departure
scheduled_arrival
200 строк получено - 0,545s (0,001s получ.), 2026-10-15 в 14:20:46


DBeaver 26.2.0 - <postgres> Script
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM flights
WHERE flights.flight_id NOT IN (
SELECT ticket_flights.flight_id
FROM ticket_flights
WHERE ticket_flights.fare_conditions = 'Business'
)
4
Результат 1
QUERY PLAN
Seq Scan on flights (cost=49821.55..51474.35 rows=32832 width=63) (actual time=934.684..963.872 rows=41705 loo
Filter: (NOT (hashed SubPlan 1))
Rows Removed by Filter: 23959
Buffers: shared hit=1121 read=19431
SubPlan 1
-> Seq Scan on ticket_flights (cost=0.00..49224.19 rows=238945 width=4) (actual time=0.163..735.922 rows=2422
Filter: ((fare_conditions)::text = 'Business'::text)
Rows Removed by Filter: 2118131
Buffers: shared hit=289 read=19431
Planning Time: 0.172 ms
Execution Time: 966.692 ms



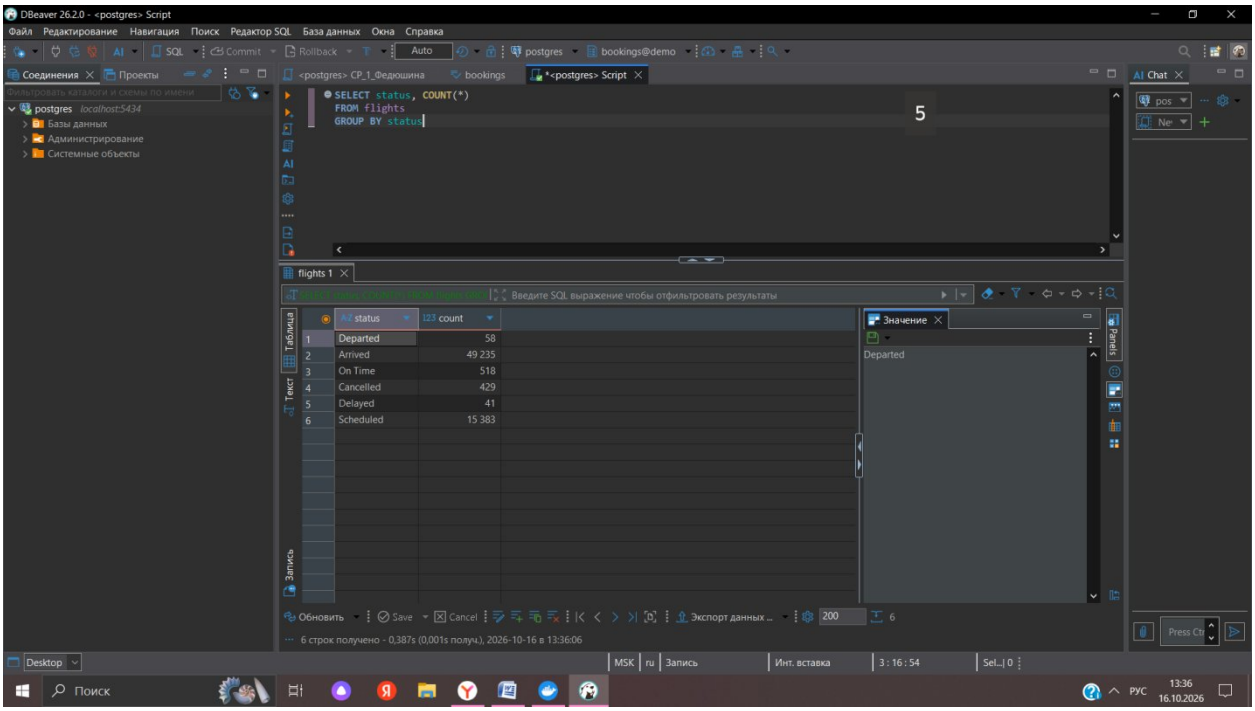


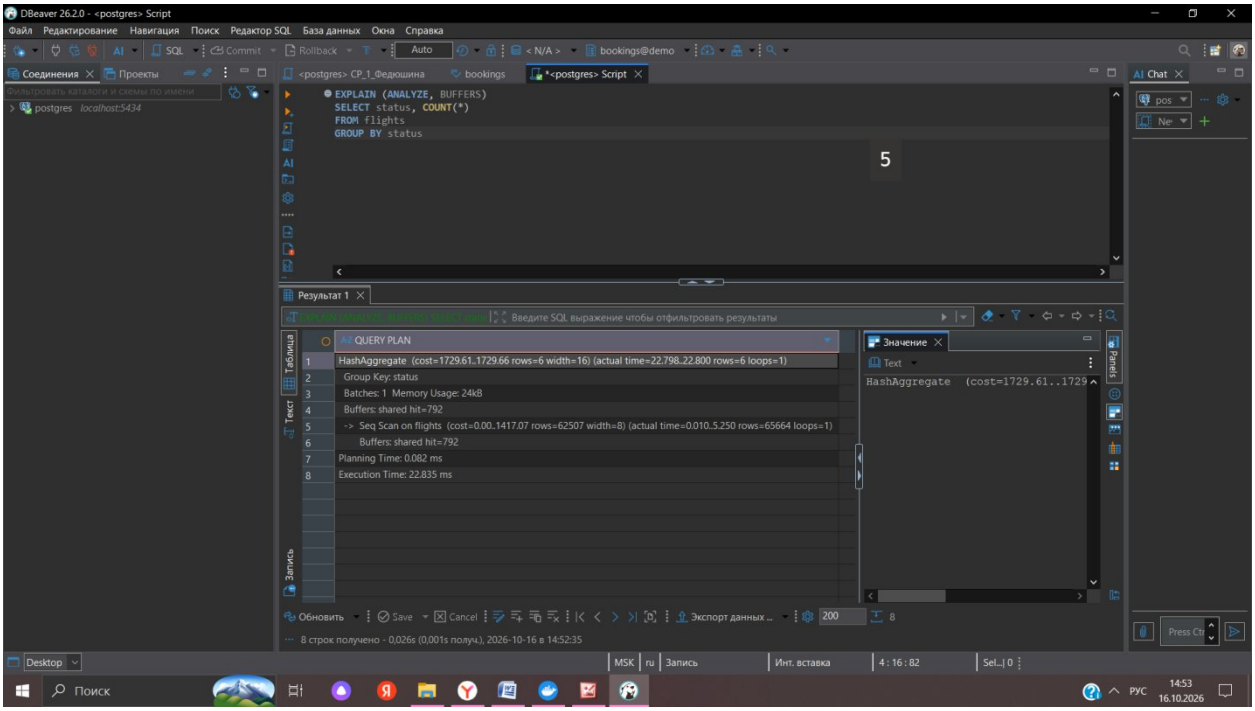
NOT IN не работает с NULL. Если в таблице есть хотя бы один NULL, то и результат окажется пустым, хотя логически рейсы без бизнес-класса должны быть. NOT EXISTS от NULL не зависит, он проверяет существование строки, а не сравнивает значения.

В первом запросе PostgreSQL использовал Seq Scan on flights + hashed SubPlan: подзапрос по ticket_flights выполнен один раз, результат захеширован, затем проверен для каждой строки flights. Во втором запросе использован Hash Right Anti Join. хеш строится по flights, а ticket_flights сканируется один раз с фильтром fare_conditions = 'Business'. Оба плана работали в основном с диском: shared read ≈ 19 400 страниц, shared hit в разы меньше; индекс не использовался ни в одном случае, везде Seq Scan. Время

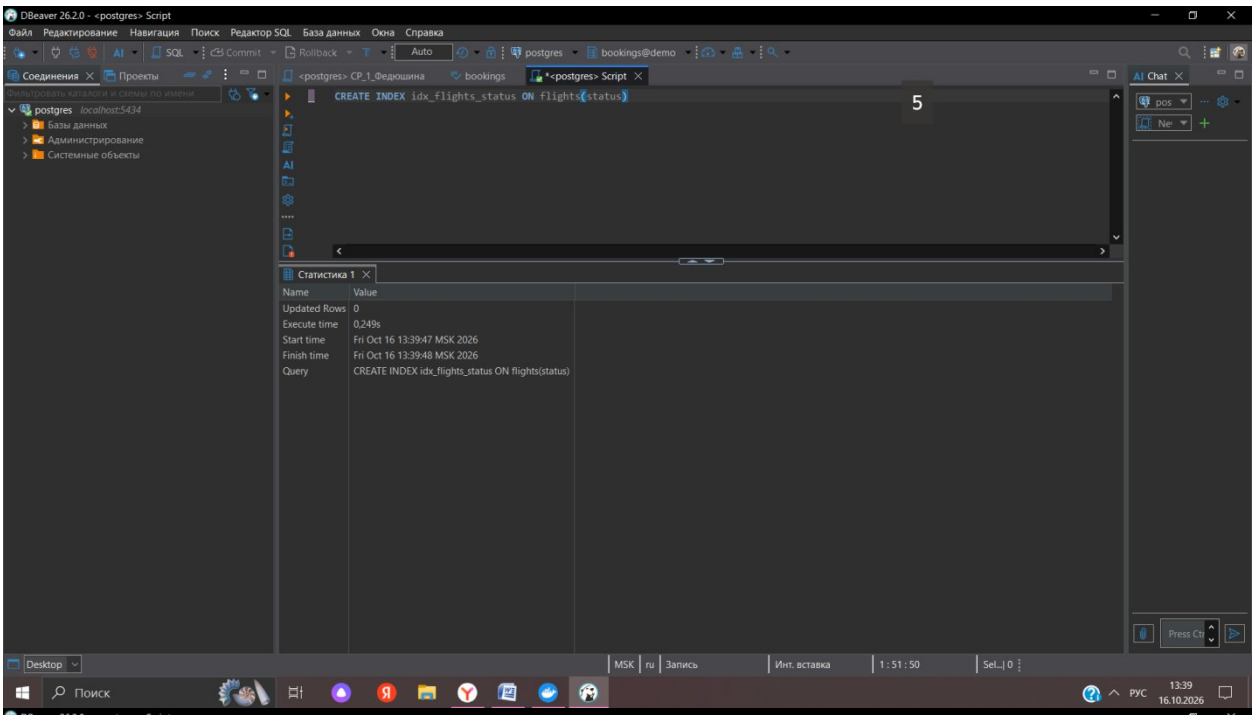
почти одинаковое 566 мс и 604 мс, но в общем случае NOT EXISTS надёжнее, так как всегда не ломается при наличии NULL в данных.

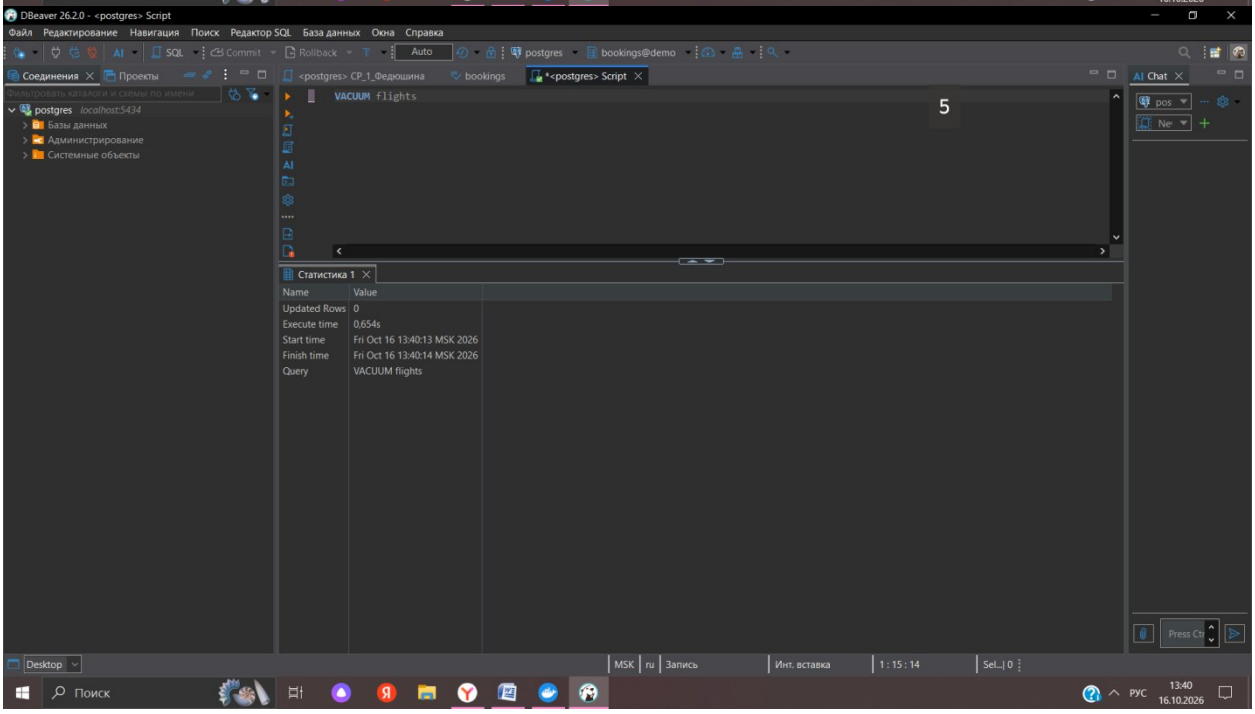
ЗАДАНИЕ 5

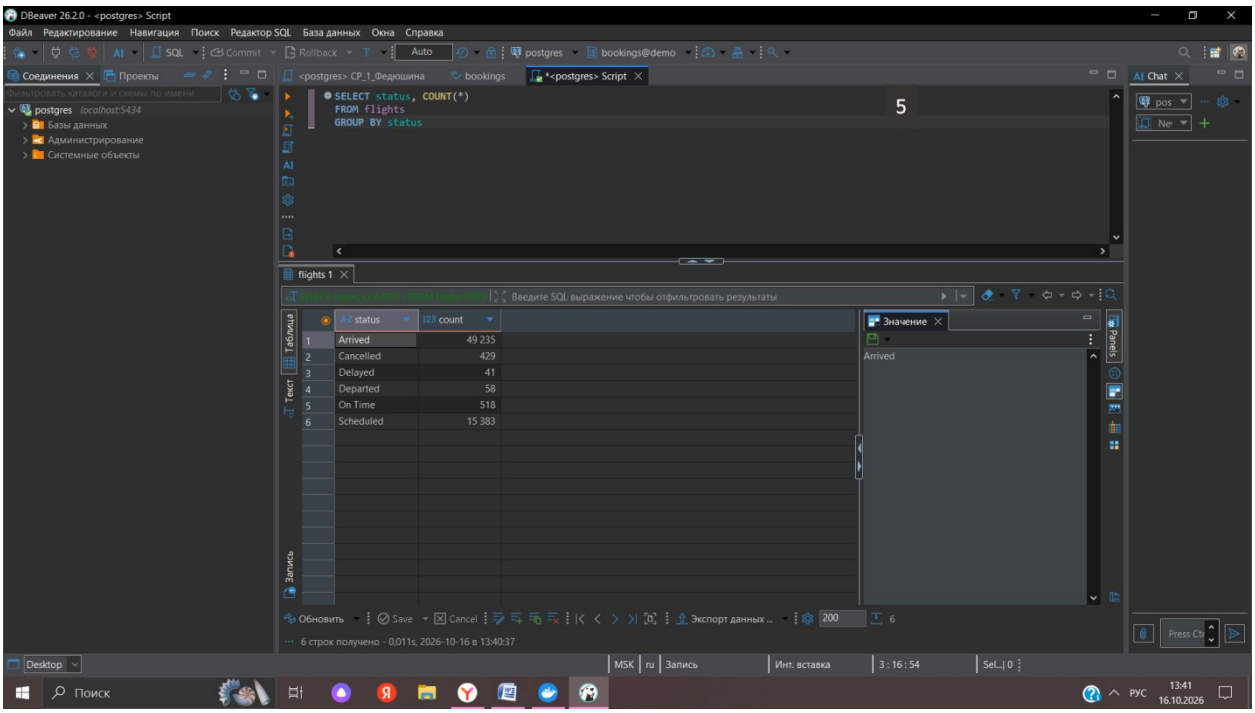


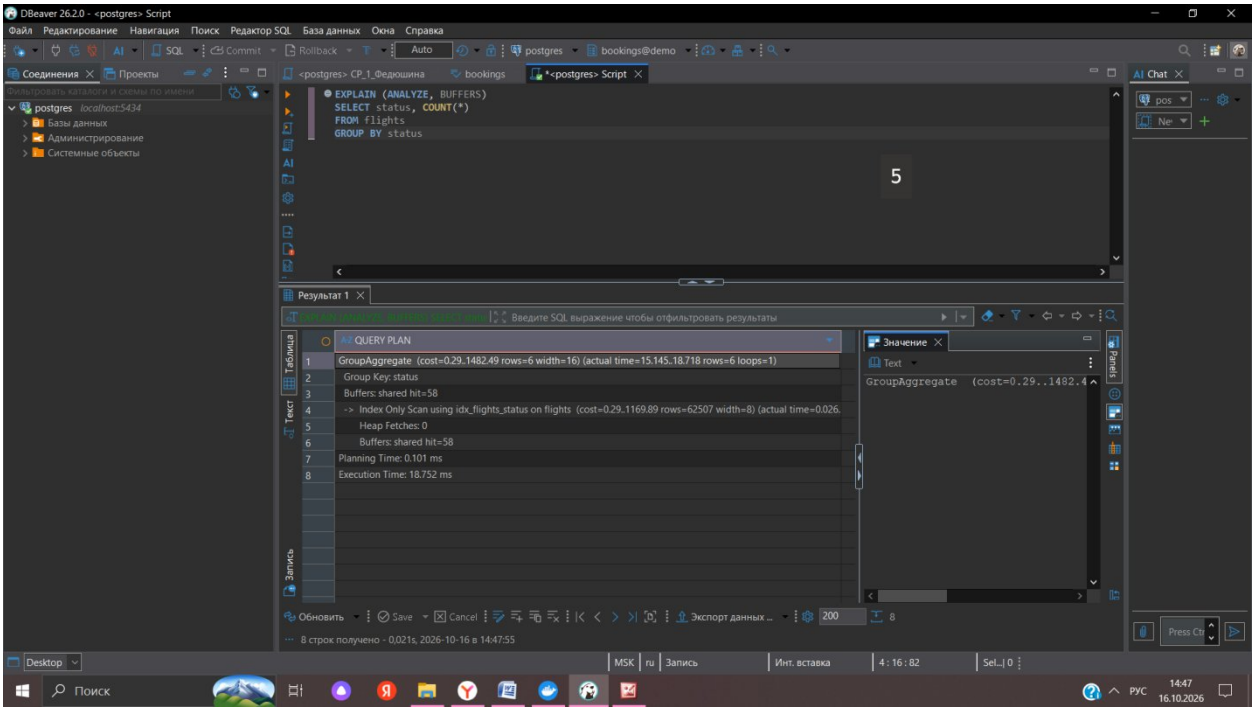


Запрос выполнялся через HashAggregate + Seq Scan on flights. Индекс не использовался, его не было. Работа шла полностью с кешем: shared hit=792, shared read=0, обращений к диску не было. Время выполнения — 22.835 мс, Buffers: shared hit=792.


DBeaver 26.2.0 - <postgres> Script
Файл Редактирование Навигация Поиск Редактор SQL База данных Окна Справка
Соединения Проекты
postgres localhost:5434
Базы данных
Администрирование
Системные объекты
<postgres> CP_1_Федюшина
bookings
*<postgres> Script
CREATE INDEX idx_flights_status ON flights(status)
5
Статистика 1
Name Value
Updated Rows 0
Execute time 0,249s
Start time Fri Oct 16 13:39:47 MSK 2026
Finish time Fri Oct 16 13:39:48 MSK 2026
Query CREATE INDEX idx_flights_status ON flights(status)
AI Chat
MSK ru Запись Инт. вставка 1 : 51 : 50
Поиск
РУС 13:39 16.10.2026


DBeaver 26.2.0 - <postgres> Script
Файл Редактирование Навигация Поиск Редактор SQL База данных Окна Справка
Соединения Проекты
postgres localhost:5434
Базы данных
Администрирование
Системные объекты
<postgres> CP_1_Федюшина
bookings
*<postgres> Script
VACUUM flights
5
Статистика 1
Name Value
Updated Rows 0
Execute time 0,654s
Start time Fri Oct 16 13:40:13 MSK 2026
Finish time Fri Oct 16 13:40:14 MSK 2026
Query VACUUM flights
AI Chat
MSK ru Запись Инт. вставка 1 : 15 : 14
Поиск
РУС 13:40 16.10.2026





После создания idx_flights_status и выполнения VACUUM flights план сменился на GroupAggregate + Index Only Scan using idx_flights_status on flights. Запрос читает данные только из индекса, без обращения к heap-таблице. Работа шла полностью с кешем: shared hit=58, shared read=0, буферов стало в 13 раз меньше (58), потому что индекс значительно меньше самой таблицы. Время выполнения сократилось до 18.752 мс.

Index Only Scan — это план, при котором PostgreSQL берёт все нужные данные прямо из индекса, не заглядывая в саму таблицу.

Index Only Scan значительно ускоряет запрос так как Меньше данных читается. Благодаря Visibility Map и VACUUM программа не тратит время на MVCC-проверку каждой строки.